



# Condiciones de Bernstein



Las **condiciones de Bernstein** son tres condiciones **teóricas** que permiten determinar si dos conjuntos de instrucciones pueden ejecutarse **concurrentemente** sin alterar el resultado del programa.

## Explicación

Para aplicar estas condiciones, tratamos las variables del programa como **conjuntos** y distinguimos entre:

- **Conjunto de lectura**  $L(S)$

Variables cuyos valores se **leen** durante la ejecución del conjunto de instrucciones  $S$ .

- **Conjunto de escritura**  $E(S)$

Variables cuyos valores se **escriben** durante la ejecución del conjunto de instrucciones  $S$ .

Dos conjuntos de instrucciones  $S_i$  y  $S_j$  pueden ejecutarse concurrentemente **si y solo si** se cumplen **simultáneamente** las tres condiciones siguientes.

## Condición Lectura–Escritura

$L(S_i) \cap E(S_j) = \emptyset$  (Read After Write – RAW)

- El conjunto de variables que  $S_i$  **lee** no debe intersectar con el conjunto de variables que  $S_j$  **escribe**.

Si se viola esta condición,  $S_i$  podría leer un valor que  $S_j$  modifica, haciendo que el resultado dependa del orden de ejecución.

```
// S1
a = b + c;

// S2
b = x + y;
```

- $L(S1) = \{ b, c \}$      $E(S1) = \{ a \}$
- $L(S2) = \{ x, y \}$      $E(S2) = \{ b \}$

Intersección relevante:

- $L(S1) \cap E(S2) = \{ b \} \neq \emptyset$

 **Conflicto:**  $S1$  lee  $b$  y  $S2$  escribe  $b$ .

**No pueden ejecutarse concurrentemente sin coordinación.**

## Condición Escritura–Lectura

$E(Si) \cap L(Sj) = \emptyset$  (Write After Read – WAR)

- El conjunto de variables que  $Si$  **escribe** no debe intersectar con el conjunto de variables que  $Sj$  **lee**.

Si se viola,  $Sj$  podría leer un valor incorrecto o incompleto dependiendo del orden de ejecución.

```
// S1
a = x + y;

// S2
b = a + z;
```

- $L(S1) = \{ x, y \}$      $E(S1) = \{ a \}$
- $L(S2) = \{ a, z \}$      $E(S2) = \{ b \}$

Intersección relevante:

- $E(S1) \cap L(S2) = \{ a \} \neq \emptyset$

● **Dependencia de datos:**  $S_2$  necesita el valor producido por  $S_1$ .

El orden importa → no concurrentes libres.

## Condición Escritura–Escritura

$E(S_i) \cap E(S_j) = \emptyset$  (Write After Write – WAW)

- Los conjuntos de variables que  $S_i$  y  $S_j$  escriben deben ser **disjuntos**.

Si ambos escriben la misma variable, el valor final dependerá del orden de ejecución.

```
// S1
a = x + y;

// S2
a = u + v;
```

- $L(S_1) = \{x, y\}$      $E(S_1) = \{a\}$
- $L(S_2) = \{u, v\}$      $E(S_2) = \{a\}$

Intersección relevante:

- $E(S_1) \cap E(S_2) = \{a\} \neq \emptyset$

● **Condición de carrera:** ambos escriben  $a$ .

Resultado no determinista sin sincronización.

## ¿Qué debemos saber?

- Si las tres intersecciones son vacías, el orden de ejecución no afecta al **resultado**.
- En ese caso, las instrucciones:
  - pueden ejecutarse concurrentemente,
  - pueden distribuirse en distintos procesadores,
  - y permiten **mejorar el rendimiento** del sistema.

- Si se viola alguna condición:
  - no pueden ejecutarse concurrentemente de forma libre,
  - aunque en un programa real podrían paralelizarse **introduciendo sincronización explícita** (locks, semáforos, etc.).